```
import numpy as np
import pandas as pd

from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression

import seaborn as sns
```

```
df = pd.read_csv('train.csv')[['Age','Pclass','SibSp','Parch','Survived']]
```

```
df
```

	Age	Pclass	SibSp	Parch	Survived
0	22.0	3	1	0	0
1	38.0	1	1	0	1
2	26.0	3	0	0	1
3	35.0	1	1	0	1
4	35.0	3	0	0	0
...	...	...	...	...	...
886	27.0	2	0	0	0
887	19.0	1	0	0	1
888	NaN	3	1	2	0
889	26.0	1	0	0	1
890	32.0	3	0	0	0

891 rows × 5 columns

Next steps: Generate code with df | New interactive sheet

```
df.dropna(inplace=True)
```

```
df
```

	Age	Pclass	SibSp	Parch	Survived
0	22.0	3	1	0	0
1	38.0	1	1	0	1
2	26.0	3	0	0	1
3	35.0	1	1	0	1
4	35.0	3	0	0	0
...	...	...	...	...	...
885	39.0	3	0	5	0
886	27.0	2	0	0	0
887	19.0	1	0	0	1
889	26.0	1	0	0	1
890	32.0	3	0	0	0

714 rows × 5 columns

Next steps: Generate code with df | New interactive sheet

```
X = df.iloc[:,0:4]
y = df.iloc[:,-1]
```

```
X
```

	Age	Pclass	SibSp	Parch
0	22.0	3	1	0
1	38.0	1	1	0
2	26.0	3	0	0
3	35.0	1	1	0
4	35.0	3	0	0
...	...	...	...	...
885	39.0	3	0	5
886	27.0	2	0	0
887	19.0	1	0	0
889	26.0	1	0	0
890	32.0	3	0	0

714 rows × 4 columns

Next steps: Generate code with X | New interactive sheet

```
y
```

	Survived
0	0
1	1
2	1
3	1
4	0
...	...
885	0
886	0
887	1
889	1
890	0

714 rows × 1 columns

dtype: int64

```
np.mean(cross_val_score(LogisticRegression(),X,y,scoring='accuracy',cv=20))
```

```
np.float64(0.6933333333333332)
```

Applying Feature Construction

```
X['Family_size'] = X['SibSp'] + X['Parch'] + 1
```

```
X
```

	Age	Pclass	SibSp	Parch	Family_size
0	22.0	3	1	0	2
1	38.0	1	1	0	2
2	26.0	3	0	0	1
3	35.0	1	1	0	2
4	35.0	3	0	0	1
...	...	...	...	...	...
885	39.0	3	0	5	6
886	27.0	2	0	0	1
887	19.0	1	0	0	1
889	26.0	1	0	0	1
890	32.0	3	0	0	1

714 rows × 5 columns

Next steps: Generate code with X New interactive sheet

```
def myfunc(num):
  if num == 1:
    #alone
    return 0
  elif num >1 and num <=4:
    # small family
    return 1
  else:
    # Large family
    return 2
```

```
myfunc(4)
```

```
1
```

```
X['Family_type'] = X['Family_size'].apply(myfunc)
```

```
X
```

	Age	Pclass	SibSp	Parch	Family_size	Family_type
0	22.0	3	1	0	2	1
1	38.0	1	1	0	2	1
2	26.0	3	0	0	1	0
3	35.0	1	1	0	2	1
4	35.0	3	0	0	1	0
...	...	...	...	...	...	...
885	39.0	3	0	5	6	2
886	27.0	2	0	0	1	0
887	19.0	1	0	0	1	0
889	26.0	1	0	0	1	0
890	32.0	3	0	0	1	0

714 rows × 6 columns

Next steps: Generate code with X New interactive sheet

```
X.drop(columns=['SibSp','Parch','Family_size'],inplace=True)
```

```
X
```

	Age	Pclass	Family_type
0	22.0	3	1
1	38.0	1	1
2	26.0	3	0
3	35.0	1	1
4	35.0	3	0
...	...	...	...
885	39.0	3	2
886	27.0	2	0
887	19.0	1	0
889	26.0	1	0
890	32.0	3	0

714 rows × 3 columns

Next steps: Generate code with X | New interactive sheet

```
np.mean(cross_val_score(LogisticRegression(),X,y,scoring='accuracy',cv=20))
```

```
np.float64(0.7003174603174602)
```

Feature Spliting

```
df = pd.read_csv('train.csv')
```

```
df
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	NaN	S
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	B42	S
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.4500	NaN	S

Next steps: Generate code with df | New interactive sheet

```
df['Name']
```

	Name
0	Braund, Mr. Owen Harris
1	Cumings, Mrs. John Bradley (Florence Briggs Th...
2	Heikkinen, Miss. Laina
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)
4	Allen, Mr. William Henry
...	...
886	Montvila, Rev. Juozas
887	Graham, Miss. Margaret Edith
888	Johnston, Miss. Catherine Helen "Carrie"
889	Behr, Mr. Karl Howell
890	Dooley, Mr. Patrick

891 rows × 1 columns

dtype: object

```
df['Title'] = df['Name'].str.split(',', expand=True)[1].str.split('.', expand=True)[0]
```

```
df['Name'].str.split(',', expand=True)[1].str.split('.', expand=True)[0]
```

	0
0	Mr
1	Mrs
2	Miss
3	Mrs
4	Mr
...	...
886	Rev
887	Miss
888	Miss
889	Mr
890	Mr

891 rows × 1 columns

dtype: object

```
df[['Title','Name']]
```

	Title	Name
0	Mr	Braund, Mr. Owen Harris
1	Mrs	Cumings, Mrs. John Bradley (Florence Briggs Th...
2	Miss	Heikkinen, Miss. Laina
3	Mrs	Futrelle, Mrs. Jacques Heath (Lily May Peel)
4	Mr	Allen, Mr. William Henry
...	...	...
886	Rev	Montvila, Rev. Juozas
887	Miss	Graham, Miss. Margaret Edith
888	Miss	Johnston, Miss. Catherine Helen "Carrie"
889	Mr	Behr, Mr. Karl Howell
890	Mr	Dooley, Mr. Patrick

891 rows × 2 columns

```
df.groupby('Title')['Survived'].mean().sort_values(ascending=False)
```

Title	Survived
Lady	1.000000
Ms	1.000000
Sir	1.000000
Mme	1.000000
the Countess	1.000000
Mlle	1.000000
Mrs	0.792000
Miss	0.697802
Master	0.575000
Major	0.500000
Col	0.500000
Dr	0.428571
Mr	0.156673
Capt	0.000000
Jonkheer	0.000000
Don	0.000000
Rev	0.000000

dtype: float64

```
import pandas as pd

df = pd.read_csv('train.csv')
df['Title'] = df['Name'].str.split(',', expand=True)[1].str.split('.', expand=True)[0]
df['Is_Married'] = 0
df.loc[df['Title'] == 'Mrs', 'Is_Married'] = 1
df['Is_Married']
```

	Is_Married
0	0
1	0
2	0
3	0
4	0
...	...
886	0
887	0
888	0
889	0
890	0

891 rows × 1 columns

dtype: int64

```
np.random.seed(23)
```

```
import numpy as np
import pandas as pd

np.random.seed(23)



mu_vec1 = np.array([0,0,0])
cov_mat1 = np.array([[1,0,0],[0,1,0],[0,0,1]])
class1_sample = np.random.multivariate_normal(mu_vec1, cov_mat1, 20)

df = pd.DataFrame(class1_sample,columns=['feature1','feature2','feature3'])
df['target'] = 1

mu_vec2 = np.array([1,1,1])
cov_mat2 =  np.array([[1,0,0],[0,1,0],[0,0,1]])
class2_sample = np.random.multivariate_normal(mu_vec2, cov_mat2, 20)

df1 = pd.DataFrame(class2_sample,columns=['feature1','feature2','feature3'])

df1['target'] = 0

df = pd.concat([df, df1], ignore_index=True)

df = df.sample(40)
```

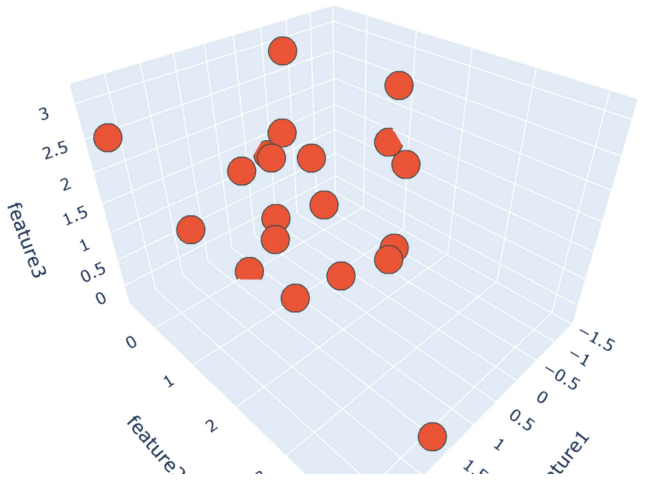
```
df
```

	feature1	feature2	feature3	target
2	-0.367548	-1.137460	-1.322148	1
34	0.177061	-0.598109	1.226512	0
14	0.420623	0.411620	-0.071324	1
11	1.968435	-0.547788	-0.679418	1
12	-2.506230	0.146960	0.606195	1
29	1.425140	1.441152	0.182561	0
31	2.224431	0.230401	1.192120	0
4	0.322272	0.060343	-1.043450	1
32	-0.723253	1.461259	-0.085367	0
33	2.823378	-0.332863	2.637391	0
36	-1.389866	0.666726	1.343517	0
18	-0.331617	-1.632386	0.619114	1
0	0.666988	0.025813	-0.777619	1
5	-1.009942	0.441736	1.128877	1
39	0.384865	1.323546	-0.103193	0
22	1.676860	4.187503	-0.080565	0
8	0.241106	-0.952510	-0.136267	1
15	-0.045438	1.040886	-0.094035	1
19	-0.992574	-0.161346	1.192404	1
30	1.437892	1.099723	1.065406	0
25	0.290746	0.866975	0.982643	0
21	0.731858	0.517441	2.244610	0
37	-1.027861	1.131416	2.603234	0
16	-0.420844	-0.551989	-0.121098	1
38	-0.764314	1.566504	1.548788	0
27	2.011059	1.920996	2.933090	0
26	0.898907	0.435960	0.820964	0
23	1.010229	1.437830	2.327788	0
1	0.948634	0.701672	-1.051082	1
10	1.415320	0.457711	0.728876	1
20	1.250737	0.186384	1.703624	0
28	0.204637	-0.011535	3.150780	0
24	0.748855	2.593111	1.170818	0
7	1.045371	0.538162	0.812119	1
6	-1.838068	-0.938769	-0.201841	1
17	0.190141	0.512137	0.131538	1
13	-0.022539	0.013422	0.935945	1
9	1.267248	0.173634	-1.223255	1
35	1.233898	0.052778	-0.261576	0
3	1.772258	-0.347459	0.670140	1

Next steps: Generate code with df | New interactive sheet

```
import plotly.express as px
#y_train_trf = y_train.astype(str)
fig = px.scatter_3d(df, x=df['feature1'], y=df['feature2'], z=df['feature3'],
                    color=df['target'].astype('str'))
fig.update_traces(marker=dict(size=12,
```

```
                                line=dict(width=2,
                                          color='DarkSlateGrey')),
                    selector=dict(mode='markers'))

fig.show()
```




```
# step 1 - Apply standard scaling
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()

df.iloc[:,0:3] = scaler.fit_transform(df.iloc[:,0:3])
```

```
# step 2 - Find Covariance Matrix
covariance_matrix = np.cov([df.iloc[:,0],df.iloc[:,1],df.iloc[:,2]])
print('Covariance Matrix:\n', covariance_matrix)
```

```
Covariance Matrix:
 [[1.02564103 0.20478114 0.080118  ]
 [0.20478114 1.02564103 0.19838882]
 [0.080118   0.19838882 1.02564103]]
```

```
# step 3 -Finding EV and EVs
eigen_values, eigen_vectors = np.linalg.eig(covariance_matrix)
```

```
eigen_values
```

```
array([1.3536065 , 0.94557084, 0.77774573])
```

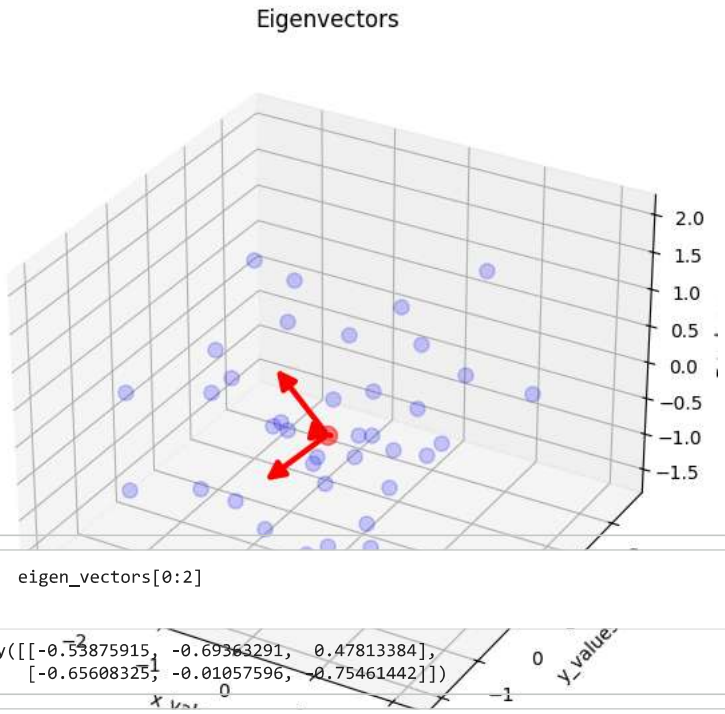
```
eigen_vectors
```

```
array([[-0.53875915, -0.69363291,  0.47813384],
       [-0.65608325, -0.01057596, -0.75461442],
       [-0.52848211,  0.72025103,  0.44938304]])
```

```
%pylab inline

from matplotlib import pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from mpl_toolkits.mplot3d import proj3d
from matplotlib.patches import FancyArrowPatch
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler

# Re-create df_synthetic with synthetic data
```

```
np.random.seed(23)
mu_vec1 = np.array([0,0,0])
cov_mat1 = np.array([[1,0,0],[0,1,0],[0,0,1]])
class1_sample = np.random.multivariate_normal(mu_vec1, cov_mat1, 20)
df_synthetic = pd.DataFrame(class1_sample,columns=['feature1','feature2','feature3'])
df_synthetic['target'] = 1

mu_vec2 = np.array([1,1,1])
cov_mat2 =  np.array([[1,0,0],[0,1,0],[0,0,1]])
class2_sample = np.random.multivariate_normal(mu_vec2, cov_mat2, 20)
df1 = pd.DataFrame(class2_sample,columns=['feature1','feature2','feature3'])
df1['target'] = 0
df_synthetic = pd.concat([df_synthetic, df1], ignore_index=True)
df_synthetic = df_synthetic.sample(40)

# Apply standard scaling
scaler = StandardScaler()
df_synthetic.iloc[:,0:3] = scaler.fit_transform(df_synthetic.iloc[:,0:3])

# Find Covariance Matrix
covariance_matrix = np.cov([df_synthetic.iloc[:,0],df_synthetic.iloc[:,1],df_synthetic.iloc[:,2]])

# Finding EV and EVs
eigen_values, eigen_vectors = np.linalg.eig(covariance_matrix)

class Arrow3D(FancyArrowPatch):
  def __init__(self, xs, ys, zs, *args, **kwargs):
    FancyArrowPatch.__init__(self, (0,0), (0,0), *args, **kwargs)
    self._verts3d = xs, ys, zs

  def draw(self, renderer):
    xs3d, ys3d, zs3d = self._verts3d
    # Correctly access the projection matrix from self.axes
    xs, ys, zs = proj3d.proj_transform(xs3d, ys3d, zs3d, self.axes.M)
    self.set_positions((xs[0],ys[0]),(xs[1],ys[1]))
    FancyArrowPatch.draw(self, renderer)

  def do_3d_projection(self, renderer=None):
    xs3d, ys3d, zs3d = self._verts3d
    if self.axes is None:
        return 0.0
    _, _, zs_projected = proj3d.proj_transform(xs3d, ys3d, zs3d, self.axes.M)
    return np.min(zs_projected)

fig = plt.figure(figsize=(7,7))
ax = fig.add_subplot(111, projection='3d')

ax.plot(df_synthetic['feature1'], df_synthetic['feature2'], df_synthetic['feature3'], 'o', markersize=8, color='blue', alpha=0.2
ax.plot([df_synthetic['feature1'].mean()], [df_synthetic['feature2'].mean()], [df_synthetic['feature3'].mean()], 'o', markersize=

mean_f1 = df_synthetic['feature1'].mean()
mean_f2 = df_synthetic['feature2'].mean()
mean_f3 = df_synthetic['feature3'].mean()

for v in eigen_vectors.T:
  a = Arrow3D([mean_f1, mean_f1 + v[0]],
              [mean_f2, mean_f2 + v[1]],
              [mean_f3, mean_f3 + v[2]],
              mutation_scale=20, lw=3, arrowstyle="-|>", color="r")
  ax.add_artist(a)
ax.set_xlabel('x_values')
ax.set_ylabel('y_values')
ax.set_zlabel('z_values')

plt.title('Eigenvectors')

plt.show()
```

```
Populating the interactive namespace from numpy and matplotlib
```




```
pc =  eigen_vectors[0:2]
pc
```

```
array([[-0.53875915, -0.69363291,  0.47813384],
       [-0.65608325, -0.01057596, -0.75461442]])
```

```
transformed_df = np.dot(df.iloc[:,0:3],pc.T)

new_df = pd.DataFrame(transformed_df,columns=['PC1','PC2'])
new_df['target'] = df['target'].values
new_df.head()
```

	PC1	PC2	target
0	0.599433	1.795862	1